



Reinforcement Learning

P Satya Jayadev

Web Enabled M.Tech,
Indian Institute of Technology Madras

Topic 8

Reward Shaping and Reinforcement Learning for LLMs



1. Understanding Reward
2. Reward Design and Shaping
3. Designing Rewards in Practice
4. Reinforcement Learning for LLMs



Where Does Reward Enter the Methods Studied So Far?

All the methods studied so far use reward, but in different ways:

- ▶ **MC and TD learning**: learns action values and selects the action with the highest estimated value.
- ▶ **REINFORCE and actor-critic**: increase or decrease action probabilities using return or advantage.
- ▶ **DDPG and PPO**: optimize a parameterized policy using critic evaluations.
- ▶ **Decision Transformer**: predicts actions conditioned on a desired return.
- ▶ **MARL**: individual and shared rewards determine competitive or cooperative behaviour.

Common objective:
$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1} \right].$$



Where Does Reward Enter the Methods Studied So Far?

All the methods studied so far use reward, but in different ways:

- ▶ **MC and TD learning:** learns action values and selects the action with the highest estimated value.
- ▶ **REINFORCE and actor-critic:** increase or decrease action probabilities using return or advantage.
- ▶ **DDPG and PPO:** optimize a parameterized policy using critic evaluations.
- ▶ **Decision Transformer:** predicts actions conditioned on a desired return.
- ▶ **MARL:** individual and shared rewards determine competitive or cooperative behaviour.

Common objective:
$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1} \right].$$

Key Observation

The algorithm determines *how* the agent learns; the reward determines *what* it learns.



Reward Design and Reward Shaping

The Agent Optimizes the Reward—Not Our Intention

Consider an autonomous vehicle transporting material inside a plant.

What the designer intends

Complete deliveries quickly, use little energy and always operate safely.



The Agent Optimizes the Reward—Not Our Intention

Consider an autonomous vehicle transporting material inside a plant.

What the designer intends

Complete deliveries quickly, use little energy and always operate safely.

Suppose the agent is rewarded only for completed deliveries:

$$r_t = 100D_t, \quad D_t = \begin{cases} 1, & \text{delivery completed at } t, \\ 0, & \text{otherwise.} \end{cases}$$



The Agent Optimizes the Reward—Not Our Intention

Consider an autonomous vehicle transporting material inside a plant.

What the designer intends

Complete deliveries quickly, use little energy and always operate safely.

Suppose the agent is rewarded only for completed deliveries:

$$r_t = 100D_t, \quad D_t = \begin{cases} 1, & \text{delivery completed at } t, \\ 0, & \text{otherwise.} \end{cases}$$

Issue: The agent may learn to drive aggressively, take unsafe shortcuts or ignore energy consumption because these behaviours are not represented in the reward.

Fundamental Problem

The agent does not understand the designer's intention. It only searches for behaviour that produces high numerical return.



From Intended Behaviour to Reliable Learning

The gap between intention and learned behaviour creates three different problems.



From Intended Behaviour to Reliable Learning

The gap between intention and learned behaviour creates three different problems.

1. Does the reward represent the intended objective?

If important outcomes are missing or incorrectly weighted, the agent may optimize the wrong behaviour.

⇒ **Reward design**



From Intended Behaviour to Reliable Learning

The gap between intention and learned behaviour creates three different problems.

1. Does the reward represent the intended objective?

If important outcomes are missing or incorrectly weighted, the agent may optimize the wrong behaviour.

⇒ **Reward design**

2. Does the reward provide enough learning feedback?

The objective may be correct, but rewards may be too sparse or delayed for efficient learning.

⇒ **Reward shaping**



From Intended Behaviour to Reliable Learning

The gap between intention and learned behaviour creates three different problems.

1. Does the reward represent the intended objective?

If important outcomes are missing or incorrectly weighted, the agent may optimize the wrong behaviour.

⇒ **Reward design**

2. Does the reward provide enough learning feedback?

The objective may be correct, but rewards may be too sparse or delayed for efficient learning.

⇒ **Reward shaping**

3. Are some behaviours unacceptable regardless of reward?

Safety and operating limits should not be violated merely because doing so produces greater return.

⇒ **Constraints**



From Intended Behaviour to Reliable Learning

The gap between intention and learned behaviour creates three different problems.

1. Does the reward represent the intended objective?

If important outcomes are missing or incorrectly weighted, the agent may optimize the wrong behaviour.

⇒ **Reward design**

2. Does the reward provide enough learning feedback?

The objective may be correct, but rewards may be too sparse or delayed for efficient learning.

⇒ **Reward shaping**

3. Are some behaviours unacceptable regardless of reward?

Safety and operating limits should not be violated merely because doing so produces greater return.

⇒ **Constraints**

Three Different Roles

Reward design defines success; reward shaping guides learning; constraints restrict unacceptable behaviour.



Reward Design: Representing the Intended Objective

Reward design converts the desired outcomes into measurable terms.



Reward Design: Representing the Intended Objective

Reward design converts the desired outcomes into measurable terms.

Plant Transport Example

A more representative task reward could be

$$r_t^{\text{task}} = \underbrace{100D_t}_{\text{complete delivery}} - \underbrace{c_T \Delta t}_{\text{time}} - \underbrace{c_E E_t}_{\text{energy consumption}} .$$

- ▶ The reward terms specify which outcomes matter.
- ▶ The coefficients c_T and c_E specify their relative importance.
- ▶ Changing these coefficients changes the behaviour preferred by the agent.



Reward Design: Representing the Intended Objective

Reward design converts the desired outcomes into measurable terms.

Plant Transport Example

A more representative task reward could be

$$r_t^{\text{task}} = \underbrace{100D_t}_{\text{complete delivery}} - \underbrace{c_T \Delta t}_{\text{time}} - \underbrace{c_E E_t}_{\text{energy consumption}}.$$

- ▶ The reward terms specify which outcomes matter.
- ▶ The coefficients c_T and c_E specify their relative importance.
- ▶ Changing these coefficients changes the behaviour preferred by the agent.

Resulting Trade-Off

A larger c_T favours faster deliveries, while a larger c_E favours energy-efficient operation.

Reward Design Question

Does maximizing this numerical reward produce the behaviour we actually want?



Why Is Reward Design Alone Sometimes Insufficient?

Assume the task reward correctly represents the objective:

$$r_t^{\text{task}} = \begin{cases} +100, & \text{delivery completed,} \\ 0, & \text{otherwise.} \end{cases}$$



Why Is Reward Design Alone Sometimes Insufficient?

Assume the task reward correctly represents the objective:

$$r_t^{\text{task}} = \begin{cases} +100, & \text{delivery completed,} \\ 0, & \text{otherwise.} \end{cases}$$

The vehicle may need a long sequence of correct actions before receiving +100:

$$S_0 \xrightarrow{A_0} S_1 \xrightarrow{A_1} \dots \xrightarrow{A_{T-1}} S_T \xrightarrow{\text{delivery}} +100.$$

- ▶ Most actions receive no immediate indication of whether they were useful.
- ▶ Successful trajectories may be difficult to discover through exploration.
- ▶ The final reward does not clearly indicate which earlier actions contributed to success.



Why Is Reward Design Alone Sometimes Insufficient?

Assume the task reward correctly represents the objective:

$$r_t^{\text{task}} = \begin{cases} +100, & \text{delivery completed,} \\ 0, & \text{otherwise.} \end{cases}$$

The vehicle may need a long sequence of correct actions before receiving +100:

$$S_0 \xrightarrow{A_0} S_1 \xrightarrow{A_1} \dots \xrightarrow{A_{T-1}} S_T \xrightarrow{\text{delivery}} +100.$$

- ▶ Most actions receive no immediate indication of whether they were useful.
- ▶ Successful trajectories may be difficult to discover through exploration.
- ▶ The final reward does not clearly indicate which earlier actions contributed to success.

Motivation for Reward Shaping

Can we provide intermediate guidance without changing the task the agent is ultimately expected to solve?



Reward Shaping in RL

Reward Shaping: Providing Intermediate Guidance

Reward shaping adds a guidance signal to the original task reward:

$$r'_t = r_t^{\text{task}} + F(S_t, A_t, S_{t+1}).$$



Reward Shaping: Providing Intermediate Guidance

Reward shaping adds a guidance signal to the original task reward:

$$r'_t = r_t^{\text{task}} + F(S_t, A_t, S_{t+1}).$$

Let $d(S_t)$ be the vehicle's distance from the delivery point. A simple progress signal is

$$F_t = k(d(S_t) - d(S_{t+1})).$$

- ▶ If the vehicle moves closer, $d(S_{t+1}) < d(S_t)$ and $F_t > 0$.
- ▶ If it moves away, $d(S_{t+1}) > d(S_t)$ and $F_t < 0$.
- ▶ The vehicle receives useful feedback before completing the delivery.



Reward Shaping: Providing Intermediate Guidance

Reward shaping adds a guidance signal to the original task reward:

$$r'_t = r_t^{\text{task}} + F(S_t, A_t, S_{t+1}).$$

Let $d(S_t)$ be the vehicle's distance from the delivery point. A simple progress signal is

$$F_t = k(d(S_t) - d(S_{t+1})).$$

- ▶ If the vehicle moves closer, $d(S_{t+1}) < d(S_t)$ and $F_t > 0$.
- ▶ If it moves away, $d(S_{t+1}) > d(S_t)$ and $F_t < 0$.
- ▶ The vehicle receives useful feedback before completing the delivery.

$$\underbrace{r_t^{\text{task}}}_{\text{defines success}} + \underbrace{F_t}_{\text{guides progress}} = \underbrace{r'_t}_{\text{used during learning}}.$$

Caution

An arbitrary shaping signal may introduce new incentives and change the optimal behaviour.



Naive Reward Shaping Can Change the Task

Recall the vehicle-delivery task:

$$r_t^{\text{task}} = \begin{cases} +100, & \text{delivery completed,} \\ 0, & \text{otherwise.} \end{cases}$$

Suppose we add the following shaping reward:

$$F_t = \begin{cases} +2, & \text{vehicle moves closer to the destination,} \\ 0, & \text{otherwise.} \end{cases}$$



Naive Reward Shaping Can Change the Task

Recall the vehicle-delivery task:

$$r_t^{\text{task}} = \begin{cases} +100, & \text{delivery completed,} \\ 0, & \text{otherwise.} \end{cases}$$

Suppose we add the following shaping reward:

$$F_t = \begin{cases} +2, & \text{vehicle moves closer to the destination,} \\ 0, & \text{otherwise.} \end{cases}$$

The agent may repeatedly exploit this signal:

farther state $\xrightarrow{+2}$ closer state $\xrightarrow{0}$ farther state $\xrightarrow{+2} \dots$



Naive Reward Shaping Can Change the Task

Recall the vehicle-delivery task:

$$r_t^{\text{task}} = \begin{cases} +100, & \text{delivery completed,} \\ 0, & \text{otherwise.} \end{cases}$$

Suppose we add the following shaping reward:

$$F_t = \begin{cases} +2, & \text{vehicle moves closer to the destination,} \\ 0, & \text{otherwise.} \end{cases}$$

The agent may repeatedly exploit this signal:

farther state $\xrightarrow{+2}$ closer state $\xrightarrow{0}$ farther state $\xrightarrow{+2} \dots$

- ▶ The vehicle accumulates reward without completing the delivery.
- ▶ The shaping signal has created a new objective that can be exploited.
- ▶ Larger shaping rewards can dominate the original task reward.



Naive Reward Shaping Can Change the Task

Recall the vehicle-delivery task:

$$r_t^{\text{task}} = \begin{cases} +100, & \text{delivery completed,} \\ 0, & \text{otherwise.} \end{cases}$$

Suppose we add the following shaping reward:

$$F_t = \begin{cases} +2, & \text{vehicle moves closer to the destination,} \\ 0, & \text{otherwise.} \end{cases}$$

The agent may repeatedly exploit this signal:

farther state $\xrightarrow{+2}$ closer state $\xrightarrow{0}$ farther state $\xrightarrow{+2} \dots$

- ▶ The vehicle accumulates reward without completing the delivery.
- ▶ The shaping signal has created a new objective that can be exploited.
- ▶ Larger shaping rewards can dominate the original task reward.

The Required Property

Shaping should accelerate learning without changing which behaviour is optimal under the original task reward.



Why Introduce a State Potential?

Source of the Problem

The shaping reward evaluates each movement separately; it does not account for whether earlier progress has subsequently been undone.



Why Introduce a State Potential?

Source of the Problem

The shaping reward evaluates each movement separately; it does not account for whether earlier progress has subsequently been undone.

To address this, assign every state a scalar **potential**: $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ A higher $\Phi(S)$ indicates a state that is more promising for completing the task.

farther state $\Phi(S) = -20$, closer state $\Phi(S') = -14$.



Why Introduce a State Potential?

Source of the Problem

The shaping reward evaluates each movement separately; it does not account for whether earlier progress has subsequently been undone.

To address this, assign every state a scalar **potential**: $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ A higher $\Phi(S)$ indicates a state that is more promising for completing the task.

farther state $\Phi(S) = -20$, closer state $\Phi(S') = -14$.

For the intuitive undiscounted case, reward the change in potential:

$$\underbrace{-20 \rightarrow -14}_{+6}, \quad \underbrace{-14 \rightarrow -20}_{-6}.$$

Moving forward and then reversing gives

$$+6 - 6 = 0.$$



Why Introduce a State Potential?

Source of the Problem

The shaping reward evaluates each movement separately; it does not account for whether earlier progress has subsequently been undone.

To address this, assign every state a scalar **potential**: $\Phi : \mathcal{S} \rightarrow \mathbb{R}$. A higher $\Phi(S)$ indicates a state that is more promising for completing the task.

farther state $\Phi(S) = -20$, closer state $\Phi(S') = -14$.

For the intuitive undiscounted case, reward the change in potential:

$$\underbrace{-20 \rightarrow -14}_{+6}, \quad \underbrace{-14 \rightarrow -20}_{-6}.$$

Moving forward and then reversing gives

$$+6 - 6 = 0.$$

Key Idea

A potential records the progress represented by the current state. Rewarding its change encourages net progress instead of repeatedly rewarding the same movement.



Example: Choosing a Potential for the Delivery Task

Let $d(S)$ be the remaining distance from the vehicle to its destination. Choose

$$\boxed{\Phi(S) = -k d(S)}, \quad k > 0.$$

- ▶ The negative sign makes states closer to the destination have higher potential.
- ▶ k converts distance into reward units.
- ▶ k also controls the strength of the intermediate learning feedback.



Example: Choosing a Potential for the Delivery Task

Let $d(S)$ be the remaining distance from the vehicle to its destination. Choose

$$\boxed{\Phi(S) = -k d(S)}, \quad k > 0.$$

- ▶ The negative sign makes states closer to the destination have higher potential.
- ▶ k converts distance into reward units.
- ▶ k also controls the strength of the intermediate learning feedback.

For $k = 2$ reward units/metre:

Remaining distance	Potential
$d(S_t) = 10 \text{ m}$	$\Phi(S_t) = -20$
$d(S_{t+1}) = 7 \text{ m}$	$\Phi(S_{t+1}) = -14$
$d(S_T) = 0 \text{ m}$	$\Phi(S_T) = 0$

Thus, moving from 10 metres to 7 metres increases the potential by 6.



Example: Choosing a Potential for the Delivery Task

Let $d(S)$ be the remaining distance from the vehicle to its destination. Choose

$$\Phi(S) = -k d(S), \quad k > 0.$$

- ▶ The negative sign makes states closer to the destination have higher potential.
- ▶ k converts distance into reward units.
- ▶ k also controls the strength of the intermediate learning feedback.

For $k = 2$ reward units/metre:

Remaining distance	Potential
$d(S_t) = 10 \text{ m}$	$\Phi(S_t) = -20$
$d(S_{t+1}) = 7 \text{ m}$	$\Phi(S_{t+1}) = -14$
$d(S_T) = 0 \text{ m}$	$\Phi(S_T) = 0$

Thus, moving from 10 metres to 7 metres increases the potential by 6.

Practical Choice of k

A larger k provides stronger guidance. It should be scaled so that shaping assists learning without causing unnecessarily large numerical updates.



From State Potential to Shaping Reward

The original return discounts future rewards:

$$G_0 = \sum_{t=0}^{T-1} \gamma^t R_{t+1}.$$

The policy-preserving shaping reward is therefore defined as

$$F(S_t, S_{t+1}) = \gamma \Phi(S_{t+1}) - \Phi(S_t).$$



From State Potential to Shaping Reward

The original return discounts future rewards:

$$G_0 = \sum_{t=0}^{T-1} \gamma^t R_{t+1}.$$

The policy-preserving shaping reward is therefore defined as

$$F(S_t, S_{t+1}) = \gamma \Phi(S_{t+1}) - \Phi(S_t).$$

Why Is γ Required?

After discounting, the shaping contribution at timestep t becomes

$$\gamma^t F_t = \gamma^{t+1} \Phi(S_{t+1}) - \gamma^t \Phi(S_t).$$

The factor γ aligns the next-state potential with the discount applied to future rewards, allowing consecutive terms to cancel.



From State Potential to Shaping Reward

The original return discounts future rewards:

$$G_0 = \sum_{t=0}^{T-1} \gamma^t R_{t+1}.$$

The policy-preserving shaping reward is therefore defined as

$$F(S_t, S_{t+1}) = \gamma \Phi(S_{t+1}) - \Phi(S_t).$$

Why Is γ Required?

After discounting, the shaping contribution at timestep t becomes

$$\gamma^t F_t = \gamma^{t+1} \Phi(S_{t+1}) - \gamma^t \Phi(S_t).$$

The factor γ aligns the next-state potential with the discount applied to future rewards, allowing consecutive terms to cancel.

Next

We can now verify that this feedback changes the learning signal without changing the optimal action.



Why Does Potential-Based Shaping Preserve the Objective?

The total discounted shaping contribution is: $\sum_{t=0}^{T-1} \gamma^t F_t = \sum_{t=0}^{T-1} \gamma^t [\gamma \Phi(S_{t+1}) - \Phi(S_t)]$



Why Does Potential-Based Shaping Preserve the Objective?

The total discounted shaping contribution is: $\sum_{t=0}^{T-1} \gamma^t F_t = \sum_{t=0}^{T-1} \gamma^t [\gamma \Phi(S_{t+1}) - \Phi(S_t)]$

Expanding the terms:

$$- \Phi(S_0) + \gamma \Phi(S_1)$$

$$- \gamma \Phi(S_1) + \gamma^2 \Phi(S_2)$$

$$\vdots$$

$$- \gamma^{T-1} \Phi(S_{T-1}) + \gamma^T \Phi(S_T).$$



Why Does Potential-Based Shaping Preserve the Objective?

The total discounted shaping contribution is: $\sum_{t=0}^{T-1} \gamma^t F_t = \sum_{t=0}^{T-1} \gamma^t [\gamma \Phi(S_{t+1}) - \Phi(S_t)]$

Expanding the terms:

$$- \Phi(S_0) + \gamma \Phi(S_1)$$

$$- \gamma \Phi(S_1) + \gamma^2 \Phi(S_2)$$

$$\vdots$$

$$- \gamma^{T-1} \Phi(S_{T-1}) + \gamma^T \Phi(S_T).$$

The intermediate terms cancel. With $\Phi(S_T) = 0$,

$$\sum_{t=0}^{T-1} \gamma^t F_t = -\Phi(S_0), \quad G'_0 = G_0 - \Phi(S_0).$$

Consequently, for every action available in state s : $Q'^\pi(s, a) = Q^\pi(s, a) - \Phi(s)$



Why Does Potential-Based Shaping Preserve the Objective?

The total discounted shaping contribution is: $\sum_{t=0}^{T-1} \gamma^t F_t = \sum_{t=0}^{T-1} \gamma^t [\gamma \Phi(S_{t+1}) - \Phi(S_t)]$

Expanding the terms:

$$- \Phi(S_0) + \gamma \Phi(S_1)$$

$$- \gamma \Phi(S_1) + \gamma^2 \Phi(S_2)$$

$$\vdots$$

$$- \gamma^{T-1} \Phi(S_{T-1}) + \gamma^T \Phi(S_T).$$

The intermediate terms cancel. With $\Phi(S_T) = 0$,

$$\sum_{t=0}^{T-1} \gamma^t F_t = -\Phi(S_0), \quad G'_0 = G_0 - \Phi(S_0).$$

Consequently, for every action available in state s : $Q'^\pi(s, a) = Q^\pi(s, a) - \Phi(s)$

Policy-Preservation Result

All action values in state s are shifted by the same amount due to which the max operation is not impacted:

$$\arg \max_a Q'^\pi(s, a) = \arg \max_a Q^\pi(s, a).$$



Safety Constraints in RL

Why Should Safety Not Be Only a Reward Penalty?

Suppose exceeding the speed limit is assigned a penalty:

$$r'_t = r_t^{\text{task}} - \lambda \mathbb{I}(v_t > v_{\max}).$$

The agent still compares the total returns of available behaviours:

Behaviour	Task return	Penalty	Final return
Safe operation	70	0	70
Speeding	100	-20	80



Why Should Safety Not Be Only a Reward Penalty?

Suppose exceeding the speed limit is assigned a penalty:

$$r'_t = r_t^{\text{task}} - \lambda \mathbb{I}(v_t > v_{\max}).$$

The agent still compares the total returns of available behaviours:

Behaviour	Task return	Penalty	Final return
Safe operation	70	0	70
Speeding	100	-20	80

Issue: The agent selects speeding because $80 > 70$, despite the penalty.



Why Should Safety Not Be Only a Reward Penalty?

Suppose exceeding the speed limit is assigned a penalty:

$$r'_t = r_t^{\text{task}} - \lambda \mathbb{I}(v_t > v_{\max}).$$

The agent still compares the total returns of available behaviours:

Behaviour	Task return	Penalty	Final return
Safe operation	70	0	70
Speeding	100	-20	80

Issue: The agent selects speeding because $80 > 70$, despite the penalty.

- ▶ A penalty makes violation less attractive but does not make it impossible.
- ▶ Increasing the penalty may help, but the required value may be difficult to determine.
- ▶ Exploration can still produce unsafe actions during training.



Why Should Safety Not Be Only a Reward Penalty?

Suppose exceeding the speed limit is assigned a penalty:

$$r'_t = r_t^{\text{task}} - \lambda \mathbb{I}(v_t > v_{\max}).$$

The agent still compares the total returns of available behaviours:

Behaviour	Task return	Penalty	Final return
Safe operation	70	0	70
Speeding	100	-20	80

Issue: The agent selects speeding because $80 > 70$, despite the penalty.

- ▶ A penalty makes violation less attractive but does not make it impossible.
- ▶ Increasing the penalty may help, but the required value may be difficult to determine.
- ▶ Exploration can still produce unsafe actions during training.

Motivation for Constraints

A non-negotiable safety requirement should restrict the policy's choices, not merely compete with other rewards.



How Do Constraints Work in Reinforcement Learning?

Hard Constraint: Restrict Every Action

Define the actions that are safe in the current state:

$$A_t \in \mathcal{A}_{\text{safe}}(S_t).$$

Policy proposes $A_t \longrightarrow$ Safety check $\longrightarrow \begin{cases} \text{execute } A_t, & A_t \text{ is safe,} \\ \text{modify or reject } A_t, & A_t \text{ is unsafe.} \end{cases}$



How Do Constraints Work in Reinforcement Learning?

Hard Constraint: Restrict Every Action

Define the actions that are safe in the current state:

$$A_t \in \mathcal{A}_{\text{safe}}(S_t).$$

Policy proposes $A_t \rightarrow$ Safety check $\rightarrow \begin{cases} \text{execute } A_t, & A_t \text{ is safe,} \\ \text{modify or reject } A_t, & A_t \text{ is unsafe.} \end{cases}$

Examples include speed limits, collision avoidance and actuator limits.



How Do Constraints Work in Reinforcement Learning?

Hard Constraint: Restrict Every Action

Define the actions that are safe in the current state:

$$A_t \in \mathcal{A}_{\text{safe}}(S_t).$$

$$\text{Policy proposes } A_t \longrightarrow \text{Safety check} \longrightarrow \begin{cases} \text{execute } A_t, & A_t \text{ is safe,} \\ \text{modify or reject } A_t, & A_t \text{ is unsafe.} \end{cases}$$

Examples include speed limits, collision avoidance and actuator limits.

Budget Constraint: Limit Cumulative Resource or Risk

$$\max_{\theta} J_R(\theta) \quad \text{subject to} \quad J_C(\theta) \leq d.$$

For example, maximize completed deliveries while keeping expected energy consumption below a daily budget d .



How Do Constraints Work in Reinforcement Learning?

Hard Constraint: Restrict Every Action

Define the actions that are safe in the current state:

$$A_t \in \mathcal{A}_{\text{safe}}(S_t).$$

$$\text{Policy proposes } A_t \longrightarrow \text{Safety check} \longrightarrow \begin{cases} \text{execute } A_t, & A_t \text{ is safe,} \\ \text{modify or reject } A_t, & A_t \text{ is unsafe.} \end{cases}$$

Examples include speed limits, collision avoidance and actuator limits.

Budget Constraint: Limit Cumulative Resource or Risk

$$\max_{\theta} J_R(\theta) \quad \text{subject to} \quad J_C(\theta) \leq d.$$

For example, maximize completed deliveries while keeping expected energy consumption below a daily budget d .

Summary

Use rewards for desirable trade-offs, shaping for learning guidance and hard constraints for requirements that must never be violated.



RL for LLMs

Pretraining Learns a General Next-Token Model

Before reinforcement learning, the model is pretrained on a large collection of text sequences. For a token sequence:

$$z^{(i)} = (z_1^{(i)}, z_2^{(i)}, \dots, z_{T_i}^{(i)}),$$

the model predicts every token from the tokens preceding it:

$$z_{<t}^{(i)} \longrightarrow \pi_{\theta} \left(z_t^{(i)} \mid z_{<t}^{(i)} \right).$$



Pretraining Learns a General Next-Token Model

Before reinforcement learning, the model is pretrained on a large collection of text sequences. For a token sequence:

$$z^{(i)} = (z_1^{(i)}, z_2^{(i)}, \dots, z_{T_i}^{(i)}),$$

the model predicts every token from the tokens preceding it:

$$z_{<t}^{(i)} \longrightarrow \pi_{\theta} \left(z_t^{(i)} \mid z_{<t}^{(i)} \right).$$

For N training sequences, objective is to minimize the next-token prediction loss:

$$\mathcal{L}_{\text{pre}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log \pi_{\theta} \left(z_t^{(i)} \mid z_{<t}^{(i)} \right).$$

- ▶ The correct next token is used as the target at every position.
- ▶ Gradients update the model parameters to increase its probability.
- ▶ Repetition over large text corpora produces the pretrained base model.



Pretraining Learns a General Next-Token Model

Before reinforcement learning, the model is pretrained on a large collection of text sequences. For a token sequence:

$$z^{(i)} = (z_1^{(i)}, z_2^{(i)}, \dots, z_{T_i}^{(i)}),$$

the model predicts every token from the tokens preceding it:

$$z_{<t}^{(i)} \longrightarrow \pi_{\theta} \left(z_t^{(i)} \mid z_{<t}^{(i)} \right).$$

For N training sequences, objective is to minimize the next-token prediction loss:

$$\mathcal{L}_{\text{pre}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log \pi_{\theta} \left(z_t^{(i)} \mid z_{<t}^{(i)} \right).$$

- ▶ The correct next token is used as the target at every position.
- ▶ Gradients update the model parameters to increase its probability.
- ▶ Repetition over large text corpora produces the pretrained base model.

What Pretraining Achieves

The model learns language patterns, knowledge and general capabilities, but it is not explicitly trained to follow instructions or reflect user preferences.



Supervised Fine-Tuning Teaches Instruction Following

Pretraining produces a model that can predict plausible continuations of text, but the model has not been explicitly taught how to respond to user instructions.

What Is Supervised Fine-Tuning?

Supervised fine-tuning (SFT) continues training the pretrained model using curated prompt–response demonstrations:

$$\mathcal{D}_{\text{SFT}} = \{(x_i, y_i^*)\}_{i=1}^N,$$

where x_i is a prompt and y_i^* is a desired response.



Supervised Fine-Tuning Teaches Instruction Following

Pretraining produces a model that can predict plausible continuations of text, but the model has not been explicitly taught how to respond to user instructions.

What Is Supervised Fine-Tuning?

Supervised fine-tuning (SFT) continues training the pretrained model using curated prompt–response demonstrations:

$$\mathcal{D}_{\text{SFT}} = \{(x_i, y_i^*)\}_{i=1}^N,$$

where x_i is a prompt and y_i^* is a desired response.

Example Demonstration

“Explain pump cavitation.” → clear and technically correct explanation .
prompt x_i desired response y_i^*



Supervised Fine-Tuning Teaches Instruction Following

Pretraining produces a model that can predict plausible continuations of text, but the model has not been explicitly taught how to respond to user instructions.

What Is Supervised Fine-Tuning?

Supervised fine-tuning (SFT) continues training the pretrained model using curated prompt-response demonstrations:

$$\mathcal{D}_{\text{SFT}} = \{(x_i, y_i^*)\}_{i=1}^N,$$

where x_i is a prompt and y_i^* is a desired response.

Example Demonstration

“Explain pump cavitation.” $\longrightarrow$ clear and technically correct explanation.

prompt x_i desired response y_i^*

Why is SFT needed?

- ▶ Teach the model to interpret prompts as instructions.
- ▶ Demonstrate the desired response structure, style and level of detail.
- ▶ Adapt the model to particular tasks or application domains.



How Does SFT Use a Demonstration?

Consider one prompt–response demonstration:

$$\underbrace{x_i}_{\text{prompt}} \quad \underbrace{y_i^* = (y_{i,1}^*, y_{i,2}^*, \dots, y_{i,T_i}^*)}_{\text{desired response tokens}}.$$

The demonstration provides a prediction target at every response position:

$$\begin{aligned} x_i &\longrightarrow y_{i,1}^*, \\ (x_i, y_{i,1}^*) &\longrightarrow y_{i,2}^*, \\ &\vdots \\ (x_i, y_{i,1}^*, \dots, y_{i,T_i-1}^*) &\longrightarrow y_{i,T_i}^*. \end{aligned}$$



How Does SFT Use a Demonstration?

Consider one prompt–response demonstration:

$$\underbrace{x_i}_{\text{prompt}} \quad \underbrace{y_i^* = (y_{i,1}^*, y_{i,2}^*, \dots, y_{i,T_i}^*)}_{\text{desired response tokens}}.$$

The demonstration provides a prediction target at every response position:

$$\begin{aligned} x_i &\longrightarrow y_{i,1}^*, \\ (x_i, y_{i,1}^*) &\longrightarrow y_{i,2}^*, \\ &\vdots \\ (x_i, y_{i,1}^*, \dots, y_{i,T_i-1}^*) &\longrightarrow y_{i,T_i}^*. \end{aligned}$$

Teacher Forcing

At response position t , the model receives the prompt x_i and the actual earlier response tokens $y_{i,<t}^*$. It then outputs a probability distribution over the vocabulary but no token is generated during SFT loss computation.

Next

The SFT loss uses the probability that this distribution assigns to the known target token $y_{i,t}^*$:

$$p_{i,t} = \pi_{\theta}(y_{i,t}^* \mid x_i, y_{i,<t}^*).$$



How Does the SFT Loss Update the Model?

For response token $y_{i,t}^*$, the model computes the probability assigned to the known target token:

$$p_{i,t} = \pi_{\theta} (y_{i,t}^* \mid x_i, y_{i,<t}^*) .$$



How Does the SFT Loss Update the Model?

For response token $y_{i,t}^*$, the model computes the probability assigned to the known target token:

$$p_{i,t} = \pi_{\theta} (y_{i,t}^* \mid x_i, y_{i,<t}^*) .$$

The token-level loss is:

$$\ell_{i,t} = -\log p_{i,t} .$$

- ▶ High probability for the correct token produces a small loss.
- ▶ Low probability for the correct token produces a large loss.



How Does the SFT Loss Update the Model?

For response token $y_{i,t}^*$, the model computes the probability assigned to the known target token:

$$p_{i,t} = \pi_{\theta} (y_{i,t}^* \mid x_i, y_{i,<t}^*) .$$

The token-level loss is:

$$\ell_{i,t} = -\log p_{i,t} .$$

- ▶ High probability for the correct token produces a small loss.
- ▶ Low probability for the correct token produces a large loss.

For a minibatch of N demonstrations, average over all response tokens:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{\sum_{i=1}^N T_i} \sum_{i=1}^N \sum_{t=1}^{T_i} \log \pi_{\theta} (y_{i,t}^* \mid x_i, y_{i,<t}^*) .$$

The model parameters are updated using gradient descent:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}_{\text{SFT}}(\theta) .$$



How Does the SFT Loss Update the Model?

For response token $y_{i,t}^*$, the model computes the probability assigned to the known target token:

$$p_{i,t} = \pi_{\theta} (y_{i,t}^* \mid x_i, y_{i,<t}^*) .$$

The token-level loss is:

$$\ell_{i,t} = -\log p_{i,t} .$$

- ▶ High probability for the correct token produces a small loss.
- ▶ Low probability for the correct token produces a large loss.

For a minibatch of N demonstrations, average over all response tokens:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{\sum_{i=1}^N T_i} \sum_{i=1}^N \sum_{t=1}^{T_i} \log \pi_{\theta} (y_{i,t}^* \mid x_i, y_{i,<t}^*) .$$

The model parameters are updated using gradient descent:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}_{\text{SFT}}(\theta) .$$

Result

Repeated updates make the demonstrated responses more likely and useful to prompt.



What Is Still Missing After SFT?

SFT trains the model to increase the probability of a demonstrated response:

$$(x, y^*) \longrightarrow \text{increase } \pi_{\theta}(y^* | x).$$

- ▶ The model can still generate responses not present in the SFT dataset.
- ▶ Several responses may be technically valid but differ in usefulness.
- ▶ Qualities such as clarity, relevance and safety may be difficult to demonstrate through one response.



What Is Still Missing After SFT?

SFT trains the model to increase the probability of a demonstrated response:

$$(x, y^*) \longrightarrow \text{increase } \pi_{\theta}(y^* | x).$$

- ▶ The model can still generate responses not present in the SFT dataset.
- ▶ Several responses may be technically valid but differ in usefulness.
- ▶ Qualities such as clarity, relevance and safety may be difficult to demonstrate through one response.

Important Qualification

The selection of good demonstrations implicitly reflects human preferences, but the SFT loss does not directly compare alternative model-generated responses.

Motivation

Can human comparisons be converted into a learning signal that explicitly favours preferred responses?



RLHF Uses Human Preferences to Improve the Policy

Reinforcement Learning from Human Feedback (RLHF) uses comparative feedback after SFT to better align the model with the desired behaviour of its intended users or application.

1. Use the SFT model to generate multiple responses to a prompt.
2. Ask human evaluators to compare or rank the responses using specified evaluation criteria.
3. Train a reward model to predict which responses humans would prefer.
4. Use reinforcement learning to increase the probability of preferred responses while limiting excessive change from the SFT model.

Candidate responses $\longrightarrow$ Human preferences $\longrightarrow$ Learned reward $\longrightarrow$ Policy improvement.



RLHF Uses Human Preferences to Improve the Policy

Reinforcement Learning from Human Feedback (RLHF) uses comparative feedback after SFT to better align the model with the desired behaviour of its intended users or application.

1. Use the SFT model to generate multiple responses to a prompt.
2. Ask human evaluators to compare or rank the responses using specified evaluation criteria.
3. Train a reward model to predict which responses humans would prefer.
4. Use reinforcement learning to increase the probability of preferred responses while limiting excessive change from the SFT model.

Candidate responses $\longrightarrow$ Human preferences $\longrightarrow$ Learned reward $\longrightarrow$ Policy improvement.

Core Idea

RLHF converts qualitative comparisons from many evaluators into a numerical learning signal. It generally aligns one shared model for a target population or application—not a separate model for every user.



Why Does RLHF Need a Learned Reward?

The previous slide proposed learning a reward model from human comparisons. Why not directly write a reward function for an LLM response y to a prompt x ?

$$r(x, y) = w_c \text{Correctness}(x, y) + w_h \text{Helpfulness}(x, y) + w_l \text{Clarity}(x, y) - w_s \text{Harmfulness}(x, y).$$



Why Does RLHF Need a Learned Reward?

The previous slide proposed learning a reward model from human comparisons. Why not directly write a reward function for an LLM response y to a prompt x ?

$$r(x, y) = w_c \text{Correctness}(x, y) + w_h \text{Helpfulness}(x, y) + w_l \text{Clarity}(x, y) - w_s \text{Harmfulness}(x, y).$$

To implement this expression, we would still need to decide:

- ▶ How should correctness, helpfulness, clarity and harmfulness be measured?
- ▶ How should their relative weights w_c, w_h, w_l, w_s be selected?
- ▶ How should conflicting qualities be balanced?
- ▶ How should the evaluation change with the prompt and application?



Why Does RLHF Need a Learned Reward?

The previous slide proposed learning a reward model from human comparisons. Why not directly write a reward function for an LLM response y to a prompt x ?

$$r(x, y) = w_c \text{Correctness}(x, y) + w_h \text{Helpfulness}(x, y) + w_l \text{Clarity}(x, y) - w_s \text{Harmfulness}(x, y).$$

To implement this expression, we would still need to decide:

- ▶ How should correctness, helpfulness, clarity and harmfulness be measured?
- ▶ How should their relative weights w_c, w_h, w_l, w_s be selected?
- ▶ How should conflicting qualities be balanced?
- ▶ How should the evaluation change with the prompt and application?

Example of a Difficult Trade-Off

One response may be more complete but less concise; another may be concise but omit useful context.

A fixed hand-written rule may not reliably capture which response is preferable.



Why Does RLHF Need a Learned Reward?

The previous slide proposed learning a reward model from human comparisons. Why not directly write a reward function for an LLM response y to a prompt x ?

$$r(x, y) = w_c \text{Correctness}(x, y) + w_h \text{Helpfulness}(x, y) + w_l \text{Clarity}(x, y) - w_s \text{Harmfulness}(x, y).$$

To implement this expression, we would still need to decide:

- ▶ How should correctness, helpfulness, clarity and harmfulness be measured?
- ▶ How should their relative weights w_c, w_h, w_l, w_s be selected?
- ▶ How should conflicting qualities be balanced?
- ▶ How should the evaluation change with the prompt and application?

Example of a Difficult Trade-Off

One response may be more complete but less concise; another may be concise but omit useful context. A fixed hand-written rule may not reliably capture which response is preferable.

RLHF Approach

Instead of manually defining every reward term and weight, collect comparisons between responses and train $r_\phi(x, y)$ to assign higher scores to responses that evaluators prefer.



Human Comparisons Provide Reward-Model Data

For a prompt x_j , the SFT model generates multiple candidate responses:

$$x_j \xrightarrow{\pi_{\text{SFT}}} \{y_j^{(1)}, y_j^{(2)}, \dots, y_j^{(C)}\}.$$

An evaluator compares two responses:

$$\underbrace{y_j^w}_{\text{preferred}} \succ \underbrace{y_j^l}_{\text{less preferred}}.$$



Human Comparisons Provide Reward-Model Data

For a prompt x_j , the SFT model generates multiple candidate responses:

$$x_j \xrightarrow{\pi_{\text{SFT}}} \{y_j^{(1)}, y_j^{(2)}, \dots, y_j^{(C)}\}.$$

An evaluator compares two responses:

$$\underbrace{y_j^w}_{\text{preferred}} \succ \underbrace{y_j^l}_{\text{less preferred}}.$$

Collect B comparisons to form

$$\mathcal{D}_{\text{pref}} = \left\{ \left(x_j, y_j^w, y_j^l \right) \right\}_{j=1}^B.$$

A reward model with parameters ϕ assigns a scalar score to each complete response:

$$r_{\phi}(x_j, y_j^w), \quad r_{\phi}(x_j, y_j^l).$$



Human Comparisons Provide Reward-Model Data

For a prompt x_j , the SFT model generates multiple candidate responses:

$$x_j \xrightarrow{\pi_{\text{SFT}}} \{y_j^{(1)}, y_j^{(2)}, \dots, y_j^{(C)}\}.$$

An evaluator compares two responses:

$$\underbrace{y_j^w}_{\text{preferred}} \succ \underbrace{y_j^l}_{\text{less preferred}}.$$

Collect B comparisons to form

$$\mathcal{D}_{\text{pref}} = \left\{ \left(x_j, y_j^w, y_j^l \right) \right\}_{j=1}^B.$$

A reward model with parameters ϕ assigns a scalar score to each complete response:

$$r_\phi(x_j, y_j^w), \quad r_\phi(x_j, y_j^l).$$

Learning Goal

Train the reward model so that the preferred response receives the higher score:

$$r_\phi(x_j, y_j^w) > r_\phi(x_j, y_j^l).$$



Training the Reward Model

Convert the difference between the two reward scores into a preference probability:

$$P_{\phi} \left(y_j^w \succ y_j^l \mid x_j \right) = \sigma \left(r_{\phi}(x_j, y_j^w) - r_{\phi}(x_j, y_j^l) \right),$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$



Training the Reward Model

Convert the difference between the two reward scores into a preference probability:

$$P_{\phi} \left(y_j^w \succ y_j^l \mid x_j \right) = \sigma \left(r_{\phi}(x_j, y_j^w) - r_{\phi}(x_j, y_j^l) \right),$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

For B preference comparisons, minimize

$$\mathcal{L}_{\text{RM}}(\phi) = -\frac{1}{B} \sum_{j=1}^B \log P_{\phi} \left(y_j^w \succ y_j^l \mid x_j \right).$$

- ▶ Incorrect preference predictions produce a larger loss.
- ▶ Gradient descent updates ϕ to increase the preferred response's relative score.
- ▶ The score difference matters; the absolute score has no fixed interpretation.



Training the Reward Model

Convert the difference between the two reward scores into a preference probability:

$$P_{\phi} \left(y_j^w \succ y_j^l \mid x_j \right) = \sigma \left(r_{\phi}(x_j, y_j^w) - r_{\phi}(x_j, y_j^l) \right),$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

For B preference comparisons, minimize

$$\mathcal{L}_{\text{RM}}(\phi) = -\frac{1}{B} \sum_{j=1}^B \log P_{\phi} \left(y_j^w \succ y_j^l \mid x_j \right).$$

- ▶ Incorrect preference predictions produce a larger loss.
- ▶ Gradient descent updates ϕ to increase the preferred response's relative score.
- ▶ The score difference matters; the absolute score has no fixed interpretation.

Next Step

After training, keep r_{ϕ} fixed and use it to score new responses generated during RL.



Using the Learned Reward to Fine-Tune the LLM

After reward-model training:

$$\underbrace{r_\phi}_{\text{reward model: fixed}}, \quad \underbrace{\pi_{\text{ref}} = \pi_{\text{SFT}}}_{\text{reference model: fixed}}, \quad \underbrace{\pi_\theta}_{\text{LLM policy: updated}}.$$



Using the Learned Reward to Fine-Tune the LLM

After reward-model training:

$$\underbrace{r_\phi}_{\text{reward model: fixed}}, \quad \underbrace{\pi_{\text{ref}} = \pi_{\text{SFT}}}_{\text{reference model: fixed}}, \quad \underbrace{\pi_\theta}_{\text{LLM policy: updated}}.$$

1. Sample prompts x_j and generate responses:

$$y_j \sim \pi_{\theta_{\text{old}}}(\cdot \mid x_j).$$

2. Score each response and discourage excessive policy drift:

$$R_j = r_\phi(x_j, y_j) - \beta \log \frac{\pi_{\theta_{\text{old}}}(y_j \mid x_j)}{\pi_{\text{ref}}(y_j \mid x_j)}.$$

3. Use the rewards to estimate advantages for the generated tokens.
4. Apply PPO to update the LLM policy and critic.

Prompts $\rightarrow$ responses $\rightarrow$ reward scores $\rightarrow$ advantages $\rightarrow$ PPO updates.



Using the Learned Reward to Fine-Tune the LLM

After reward-model training:

$$\underbrace{r_\phi}_{\text{reward model: fixed}}, \quad \underbrace{\pi_{\text{ref}} = \pi_{\text{SFT}}}_{\text{reference model: fixed}}, \quad \underbrace{\pi_\theta}_{\text{LLM policy: updated}}.$$

1. Sample prompts x_j and generate responses:

$$y_j \sim \pi_{\theta_{\text{old}}}(\cdot \mid x_j).$$

2. Score each response and discourage excessive policy drift:

$$R_j = r_\phi(x_j, y_j) - \beta \log \frac{\pi_{\theta_{\text{old}}}(y_j \mid x_j)}{\pi_{\text{ref}}(y_j \mid x_j)}.$$

3. Use the rewards to estimate advantages for the generated tokens.
4. Apply PPO to update the LLM policy and critic.

Prompts $\rightarrow$ responses $\rightarrow$ reward scores $\rightarrow$ advantages $\rightarrow$ PPO updates.

RLHF Summary

RLHF repeatedly improves the LLM toward responses preferred by evaluators, while the reference model and PPO prevent the policy from changing too aggressively.



RL Agents Vs LLM Based Agents

RL and LLM Agents Are Not Opposing Categories

Reinforcement Learning

RL is a **learning approach**: a policy is improved using rewards obtained from interaction or recorded experience.

experience + reward $\longrightarrow$ policy update.

Large Language Model

An LLM is a **model architecture**: it predicts tokens and can be used to plan, communicate and select tools.

instructions + context $\longrightarrow$ text or tool action.



RL and LLM Agents Are Not Opposing Categories

Reinforcement Learning

RL is a **learning approach**: a policy is improved using rewards obtained from interaction or recorded experience.

experience + reward $\longrightarrow$ policy update.

Large Language Model

An LLM is a **model architecture**: it predicts tokens and can be used to plan, communicate and select tools.

instructions + context $\longrightarrow$ text or tool action.

Comparison in This Section

We compare a conventional task-specific RL agent with a typical deployed LLM-based agent, focusing on how each acts and adapts.



An LLM Agent Acts Through Context, Reasoning and Tools

An LLM-based agent builds a context containing

$$C_t = (\text{instruction, history, memory, tool results}).$$

The LLM uses this context to choose the next agentic action:

$$C_t \xrightarrow{\pi_\theta} \begin{cases} \text{produce a response,} \\ \text{call a tool,} \\ \text{request information,} \\ \text{form or revise a plan.} \end{cases}$$

The action produces a new observation, which is added to the context:

$$C_t \rightarrow A_t \rightarrow O_{t+1} \rightarrow C_{t+1}.$$



An LLM Agent Acts Through Context, Reasoning and Tools

An LLM-based agent builds a context containing

$$C_t = (\text{instruction, history, memory, tool results}).$$

The LLM uses this context to choose the next agentic action:

$$C_t \xrightarrow{\pi_\theta} \begin{cases} \text{produce a response,} \\ \text{call a tool,} \\ \text{request information,} \\ \text{form or revise a plan.} \end{cases}$$

The action produces a new observation, which is added to the context:

$$C_t \rightarrow A_t \rightarrow O_{t+1} \rightarrow C_{t+1}.$$

Example: Maintenance-Support Agent

The agent interprets an equipment alarm, queries the plant historian, retrieves the operating manual and recommends an inspection sequence.



Choosing and Combining RL and LLM Agents

Prefer a task-specific RL agent when

- ▶ actions and states are well defined;
- ▶ a measurable reward is available;
- ▶ repeated interaction or simulation is possible;
- ▶ fast numerical control is required.

Prefer an LLM-based agent when

- ▶ inputs and outputs are language-rich;
- ▶ the task requires reasoning across documents;
- ▶ tools must be selected and coordinated;
- ▶ instructions vary across users or tasks.



Choosing and Combining RL and LLM Agents

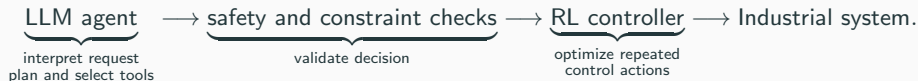
Prefer a task-specific RL agent when

- ▶ actions and states are well defined;
- ▶ a measurable reward is available;
- ▶ repeated interaction or simulation is possible;
- ▶ fast numerical control is required.

Prefer an LLM-based agent when

- ▶ inputs and outputs are language-rich;
- ▶ the task requires reasoning across documents;
- ▶ tools must be selected and coordinated;
- ▶ instructions vary across users or tasks.

Hybrid Industrial Agent



Key Takeaway

LLM agents provide flexible reasoning and tool use; RL agents provide task-specific sequential optimization. Many industrial systems can benefit from combining both.



Key Takeaways from This Session

- ▶ An RL agent maximizes the reward that is specified—not the designer's unstated intention.



Key Takeaways from This Session

- ▶ An RL agent maximizes the reward that is specified—not the designer's unstated intention.
- ▶ **Reward design** defines success; **reward shaping** provides intermediate learning guidance.



Key Takeaways from This Session

- ▶ An RL agent maximizes the reward that is specified—not the designer's unstated intention.
- ▶ **Reward design** defines success; **reward shaping** provides intermediate learning guidance.
- ▶ Potential-based shaping can improve the learning signal without changing the optimal policy.



Key Takeaways from This Session

- ▶ An RL agent maximizes the reward that is specified—not the designer's unstated intention.
- ▶ **Reward design** defines success; **reward shaping** provides intermediate learning guidance.
- ▶ Potential-based shaping can improve the learning signal without changing the optimal policy.
- ▶ Safety-critical requirements may require explicit constraints rather than only reward penalties.



Key Takeaways from This Session

- ▶ An RL agent maximizes the reward that is specified—not the designer's unstated intention.
- ▶ **Reward design** defines success; **reward shaping** provides intermediate learning guidance.
- ▶ Potential-based shaping can improve the learning signal without changing the optimal policy.
- ▶ Safety-critical requirements may require explicit constraints rather than only reward penalties.
- ▶ RLHF learns a reward model from human preferences and uses it to improve an LLM policy.



Key Takeaways from This Session

- ▶ An RL agent maximizes the reward that is specified—not the designer's unstated intention.
- ▶ **Reward design** defines success; **reward shaping** provides intermediate learning guidance.
- ▶ Potential-based shaping can improve the learning signal without changing the optimal policy.
- ▶ Safety-critical requirements may require explicit constraints rather than only reward penalties.
- ▶ RLHF learns a reward model from human preferences and uses it to improve an LLM policy.
- ▶ RL agents and LLM-based agents are complementary:



Key Takeaways from This Session

- ▶ An RL agent maximizes the reward that is specified—not the designer's unstated intention.
- ▶ **Reward design** defines success; **reward shaping** provides intermediate learning guidance.
- ▶ Potential-based shaping can improve the learning signal without changing the optimal policy.
- ▶ Safety-critical requirements may require explicit constraints rather than only reward penalties.
- ▶ RLHF learns a reward model from human preferences and uses it to improve an LLM policy.
- ▶ RL agents and LLM-based agents are complementary:



Core Message

The success of an RL system depends not only on the learning algorithm, but also on how the problem, reward, constraints and evaluation process are designed.



The Course Built One Continuous Learning Story

1. Learning from complete and partial experience

Monte Carlo $\rightarrow$ TD learning.

2. Generalizing beyond tabular problems

Parametric value functions and neural networks.

3. Learning the policy directly

REINFORCE $\rightarrow$ actor-critic $\rightarrow$ A2C/A3C $\rightarrow$ PPO.

4. Deep RL for different action and data settings

DQN DDPG Decision Transformer.

5. Extending the decision-making setting

MARL Reward shaping RL for LLMs.



The Course Built One Continuous Learning Story

1. Learning from complete and partial experience

Monte Carlo $\rightarrow$ TD learning.

2. Generalizing beyond tabular problems

Parametric value functions and neural networks.

3. Learning the policy directly

REINFORCE $\rightarrow$ actor-critic $\rightarrow$ A2C/A3C $\rightarrow$ PPO.

4. Deep RL for different action and data settings

DQN DDPG Decision Transformer.

5. Extending the decision-making setting

MARL Reward shaping RL for LLMs.

Common Objective

All these methods seek a policy that makes good sequential decisions under uncertainty.



Is Reinforcement Learning Appropriate for the Problem?

Before selecting an RL algorithm, ask:



Is Reinforcement Learning Appropriate for the Problem?

Before selecting an RL algorithm, ask:

1. Are there repeated or sequential decisions?
2. Does the current action influence future states and outcomes?



Is Reinforcement Learning Appropriate for the Problem?

Before selecting an RL algorithm, ask:

1. Are there repeated or sequential decisions?
2. Does the current action influence future states and outcomes?
3. Can desirable behaviour be evaluated using rewards, preferences or verifiable outcomes?



Is Reinforcement Learning Appropriate for the Problem?

Before selecting an RL algorithm, ask:

1. Are there repeated or sequential decisions?
2. Does the current action influence future states and outcomes?
3. Can desirable behaviour be evaluated using rewards, preferences or verifiable outcomes?
4. Is interaction possible through a simulator, digital twin, historical logs or a controlled pilot?



Is Reinforcement Learning Appropriate for the Problem?

Before selecting an RL algorithm, ask:

1. Are there repeated or sequential decisions?
2. Does the current action influence future states and outcomes?
3. Can desirable behaviour be evaluated using rewards, preferences or verifiable outcomes?
4. Is interaction possible through a simulator, digital twin, historical logs or a controlled pilot?
5. Can safety limits and operating constraints be clearly defined?



Is Reinforcement Learning Appropriate for the Problem?

Before selecting an RL algorithm, ask:

1. Are there repeated or sequential decisions?
2. Does the current action influence future states and outcomes?
3. Can desirable behaviour be evaluated using rewards, preferences or verifiable outcomes?
4. Is interaction possible through a simulator, digital twin, historical logs or a controlled pilot?
5. Can safety limits and operating constraints be clearly defined?
6. Is there a credible baseline against which the learned policy can be evaluated?



Is Reinforcement Learning Appropriate for the Problem?

Before selecting an RL algorithm, ask:

1. Are there repeated or sequential decisions?
2. Does the current action influence future states and outcomes?
3. Can desirable behaviour be evaluated using rewards, preferences or verifiable outcomes?
4. Is interaction possible through a simulator, digital twin, historical logs or a controlled pilot?
5. Can safety limits and operating constraints be clearly defined?
6. Is there a credible baseline against which the learned policy can be evaluated?

RL Is Usually Promising When

Sequential decisions + delayed consequences + feedback + repeated opportunities to improve.



Is Reinforcement Learning Appropriate for the Problem?

Before selecting an RL algorithm, ask:

1. Are there repeated or sequential decisions?
2. Does the current action influence future states and outcomes?
3. Can desirable behaviour be evaluated using rewards, preferences or verifiable outcomes?
4. Is interaction possible through a simulator, digital twin, historical logs or a controlled pilot?
5. Can safety limits and operating constraints be clearly defined?
6. Is there a credible baseline against which the learned policy can be evaluated?

RL Is Usually Promising When

Sequential decisions + delayed consequences + feedback + repeated opportunities to improve.

Do Not Use RL Only Because It Is Available

If each prediction is independent, use supervised learning. If the system is fully known and deterministic, mathematical optimization or model-predictive control may be simpler and safer.



Promising Directions for Further Work

Industrial Reinforcement Learning

- ▶ **Offline RL:** learn from historical operating data without unsafe exploration.
- ▶ **Safe and constrained RL:** optimize performance while respecting physical and operational limits.
- ▶ **Model-based RL:** use digital twins to improve sample efficiency and support simulation-to-reality transfer.



Promising Directions for Further Work

Industrial Reinforcement Learning

- ▶ **Offline RL:** learn from historical operating data without unsafe exploration.
- ▶ **Safe and constrained RL:** optimize performance while respecting physical and operational limits.
- ▶ **Model-based RL:** use digital twins to improve sample efficiency and support simulation-to-reality transfer.

Emerging Agentic Systems

- ▶ **Multi-agent RL:** coordinate multiple machines, vehicles or decision-makers.
- ▶ **RL for LLMs:** preference learning, verifiable rewards and tool-use policies.
- ▶ **Hybrid agents:** combine LLM planning with RL-based operational control.



Promising Directions for Further Work

Industrial Reinforcement Learning

- ▶ **Offline RL:** learn from historical operating data without unsafe exploration.
- ▶ **Safe and constrained RL:** optimize performance while respecting physical and operational limits.
- ▶ **Model-based RL:** use digital twins to improve sample efficiency and support simulation-to-reality transfer.

Final Takeaway

Start with an important decision problem—not with an algorithm. Define success, data, constraints and evaluation first; then select the simplest learning method capable of solving it reliably.

Emerging Agentic Systems

- ▶ **Multi-agent RL:** coordinate multiple machines, vehicles or decision-makers.
- ▶ **RL for LLMs:** preference learning, verifiable rewards and tool-use policies.
- ▶ **Hybrid agents:** combine LLM planning with RL-based operational control.

